

การใช้หน่วยความจำ ROM เป็นคลังคำสั่งควบคุม

(Lab 1 : ROM as Control Store)

ด้วยโปรแกรม Logisim-Evolution

การออกแบบหน่วยควบคุมแบบ Microprogram (Microprogrammed Control Unit) ที่เก็บลำดับสัญญาณควบคุมไว้ใน ROM และขับเคลื่อนด้วย Microsequencer

จัดทำโดย

- | | | |
|---|---------------------------|------------|
| 1 | นายตรัณภพ พิทักษ์กิจไพศาล | 6710301007 |
| 2 | นายอภิสิทธิ์ คงรักดี | 6710301009 |
| 3 | นายธนัท จงธีรธนโชติ | 6710301032 |

เสนอ อาจารย์ศุภาเทพ สวัสดิพิศาล

รายวิชา Embedded System Design ภาคการศึกษาที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา Embedded System Design ภาคการศึกษาที่ 1 ปีการศึกษา 2569 จัดทำขึ้นเพื่อนำเสนอผลการวิเคราะห์และทดสอบวงจร **Lab 1 : ROM as Control Store** บนโปรแกรม Logisim-Evolution ซึ่งเป็นการสร้างหน่วยควบคุมขนาดเล็กที่ใช้หน่วยความจำ ROM ทำหน้าที่เป็น “คลังคำสั่งควบคุม” (Control Store) แทนการต่อวงจรลอจิกควบคุมด้วย Gate แบบตายตัว อันเป็นแนวคิดพื้นฐานของการออกแบบหน่วยประมวลผลแบบ Microprogram

เนื้อหาภายในรายงานประกอบด้วยแนวคิดการออกแบบหน่วยควบคุมทั้งสองแนวทาง รายการอุปกรณ์ทั้งหมดที่ใช้ภายในวงจร รูปแบบของ Microinstruction ขนาด 8 Bit การทำงานของแต่ละส่วนโดยละเอียด ได้แก่ ส่วนลำดับที่อยู่ (MUX, μ PC และ Adder) ตัวคลังคำสั่ง (ROM) วงจรถอดรหัส (Splitter) ตัวนับ (CNT) วงจรตัดสินใจกระโดด (Branch Logic) และส่วนแสดงผล ตลอดจนการถอดรหัสโปรแกรมที่บรรจุอยู่ใน ROM ตารางการทำงานทีละคาบสัญญาณนาฬิกาทั้ง 42 คาบ วิธีการทดลองในโปรแกรม และข้อสังเกตที่ควรระวัง

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้ที่สนใจศึกษาการออกแบบหน่วยควบคุมแบบ Microprogram หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้ และขอขอบพระคุณอาจารย์ศุภดา เทพ สวัสดิพิศาล ที่ให้คำแนะนำตลอดการทำรายงานฉบับนี้

คณะผู้จัดทำ

สารบัญ

1	บทนำ	5
2	วัตถุประสงค์	5
3	เครื่องมือและอุปกรณ์ที่ใช้	5
3.1	โปรแกรมจำลองการทำงาน	5
3.2	รายการอุปกรณ์ทั้งหมดในวงจร	6
4	แนวคิดการออกแบบวงจร	7
4.1	หน่วยควบคุมสองแนวทาง	7
4.2	BlockDiagram รวมของวงจร	7
4.3	รูปแบบของ Microinstruction (8 Bit)	8
5	การทำงานของแต่ละส่วน	9
5.1	ส่วนลำดับที่อยู่: MUX + μ PC + Adder	9
5.2	ROM — ตัวคลั่งคำสั่ง	9
5.3	Splitter — ตัวถอดรหัสคำสั่ง	9
5.4	ตัวนับ CNT — งานจริงที่โปรแกรมสั่ง	10
5.5	Comparator + Branch Logic	10
5.6	Tri-state Buffer + Hex Display	11
5.7	สัญญาณนาฬิกาและการ Reset	11
6	โปรแกรมที่บรรจุอยู่ใน ROM	11
6.1	ตารางถอดรหัส Microprogram	11
6.2	ผังการไหลของโปรแกรม	12
7	ตารางการทำงานที่ละคาบสัญญาณนาฬิกา	12
8	วิธีทดลองใน Logisim-Evolution	13
8.1	การกำหนดแหล่งสัญญาณนาฬิกาก่อนใช้ Auto-Tick	14
9	ข้อสังเกตและจุดที่ควรระวัง	16
10	สรุปผล	17
11	เอกสารอ้างอิง	18

สารบัญตาราง

ตารางที่ 1	รายการอุปกรณ์ทั้งหมดในวงจรและหน้าที่	6
ตารางที่ 2	ความหมายของแต่ละ Bit ใน Microinstruction	8
ตารางที่ 3	การอ่านความหมายของวงจรตัดสินใจระดับ Gate	10
ตารางที่ 4	การถอดรหัส Microprogram ที่บรรจุใน ROM	11
ตารางที่ 5	การทำงานที่ละคาบสัญญาณนาฬิกา (42 คาบ)	12

สารบัญรูปภาพ

รูปที่ 1	BlockDiagram รวมของวงจร ROM as Control Store	8
รูปที่ 2	รูปแบบ Bit ของ Microinstruction	8
รูปที่ 3	วงจรตัดสินใจระดับ Gate	10

รูปที่ 4 ผังการไหลของ Microprogram	12
รูปที่ 5 วงจรรวมบนพื้นที่ทำงานของ Logisim-Evolution ในสถานะเริ่มต้น	14
รูปที่ 6 หน้าต่าง Hex Editor สำหรับแก้ไขข้อมูลภายใน ROM	14
รูปที่ 7 หน้าต่าง Clock Source Selection	15
รูปที่ 8 ผลการจำลองเมื่อโปรแกรมหยุดค้างที่ค่า A	16

ในหน่วยประมวลผลกลาง (CPU) ทุกชนิด **หน่วยควบคุม (Control Unit)** คือส่วนที่ทำหน้าที่สร้างสัญญาณควบคุมไปสั่งงานส่วนต่าง ๆ ของวงจร เช่น สั่งให้ Register โหลดค่า สั่งให้ ALU บวก หรือสั่งให้ตัวนับเพิ่มค่า การสร้างหน่วยควบคุมสามารถทำได้ 2 แนวทางหลัก ได้แก่ การต่อวงจรลอจิกด้วย Gate โดยตรง (Hardwired Control) และการเก็บลำดับของสัญญาณควบคุมไว้ในหน่วยความจำแล้วอ่านออกมาทีละคำ (Microprogrammed Control)

รายงานฉบับนี้นำเสนอผลการวิเคราะห์วงจร **Lab 1 : ROM as Control Store** ซึ่งเป็นหน่วยควบคุมแบบ Microprogram ขนาดเล็กที่ใช้ ROM ขนาด 16×8 Bit ทำหน้าที่เป็นคลังคำสั่งควบคุม โดยในทุกคาบสัญญาณนาฬิกา วงจรจะอ่านคำสั่งขนาด 8 Bit จาก ROM ออกมาหนึ่งคำ นำ 4 Bit บนไปใช้เป็นสัญญาณควบคุม และนำ 4 Bit ล่างไปใช้เป็นที่อยู่ปลายทางของการกระโดด จากนั้นจึงตัดสินใจคำสั่งถัดไปคือ “ที่อยู่ถัดไป (PC + 1)” หรือ “กระโดดไปที่อยู่ปลายทาง”

โปรแกรมที่บรรจุอยู่ใน ROM ทำหน้าที่นับเลขตั้งแต่ 0 ถึง 10 แสดงผลบนจอแสดงผล 7-segment แบบเลขฐานสิบหก แล้วหยุดค้างที่ค่า A (เท่ากับ 10 ในฐานสิบ) โดยใช้เวลาดังกล่าว 41 ขอบสัญญาณนาฬิกา การวิเคราะห์ในรายงานนี้จัดทำโดยการอ่านไฟล์วงจรโดยตรง ตรวจสอบตำแหน่งขาสัญญาณทุกตัวด้วยการจำลองจริง และจำลองการทำงานที่ละคาบสัญญาณนาฬิกาเพื่อยืนยันความถูกต้อง

ประเด็นสำคัญของการทดลองนี้ — วงจรแบบ Microprogram สามารถเปลี่ยนพฤติกรรมของ Hardware ได้โดยการแก้ไขเพียงข้อมูลใน ROM เท่านั้น **ไม่ต้องแกะต้อง** สายไฟแม้แต่เส้นเดียว ซึ่งเป็นข้อได้เปรียบหลักเหนือการต่อวงจรควบคุมแบบตายตัว

1. เพื่อศึกษาแนวคิดของหน่วยควบคุมแบบ Microprogram (Microprogrammed Control Unit) และเปรียบเทียบกับหน่วยควบคุมแบบต่อวงจรตายตัว (Hardwired Control)
2. เพื่อทำความเข้าใจบทบาทของ ROM ในฐานะคลังคำสั่งควบคุม (Control Store) และโครงสร้างของ Microsequencer (Microsequencer) ที่ประกอบด้วย MUX, μPC และ Adder
3. เพื่อออกแบบและตีความรูปแบบของ Microinstruction ขนาด 8 Bit ที่แบ่งเป็นสัญญาณควบคุม 4 Bit และที่อยู่ปลายทาง 4 Bit
4. เพื่อวิเคราะห์วงจรตัดสินใจกระโดด (Branch Logic) ที่สร้างจาก NOT Gate, OR และ AND ตลอดจนสมการลอจิกของสัญญาณเลือก (SEL)
5. เพื่อศึกษาการใช้งาน Tri-state Buffer ในการจำลอง Shared Bus (Shared Bus) และการควบคุมการแสดงผลด้วยสัญญาณจากหน่วยควบคุม
6. เพื่อฝึกการจำลองการทำงานที่ละคาบสัญญาณนาฬิกาในโปรแกรม Logisim-Evolution และตรวจสอบความถูกต้องของวงจรตลอดทั้ง 42 คาบ

3.1 โปรแกรมจำลองการทำงาน

- **Logisim-Evolution v4.1.0** — โปรแกรมสำหรับออกแบบและทดสอบวงจรลอจิก ใช้สร้างวงจรหน่วยควบคุมและจำลองการทำงานที่ละคาบสัญญาณนาฬิกา ไฟล์วงจรที่ใช้ในรายงานนี้คือ **Lab_1_ROM_as_Control_Store.circ**
- **เครื่องมือ Poke Tool** — ใช้กดสลับค่าที่ขา Clock และ Reset เพื่อเดินวงจรที่ละคาบและสังเกตผลลัพธ์

- **Function Auto-Tick** — ใช้เดินสัญญาณนาฬิกาอัตโนมัติเพื่อการทำงานต่อเนื่องตั้งแต่ต้นจนโปรแกรมหยุด โดยต้องกำหนดแหล่งสัญญาณนาฬิกาให้เป็นขา Clock ก่อน (ดูหัวข้อ 8.1)

3.2 รายการอุปกรณ์ทั้งหมดในวงจร

วงจรทั้งหมดถูกสร้างขึ้นภายในวงจรเดียว (ไม่มีการแบ่งเป็นวงจรย่อย) โดยมีอุปกรณ์และการตั้งค่าตามที่ตรวจสอบได้จากไฟลวงจรโดยตรง สรุปไว้ในตารางที่ 1

ตารางที่ 1 รายการอุปกรณ์ทั้งหมดในวงจรและหน้าที่

อุปกรณ์	พิกัดในไฟล์	การตั้งค่า	หน้าที่
ROM	(440, 120)	16 × 8 Bit	คลังคำสั่งควบคุม เก็บ Microprogram 6 คำสั่ง
Register (μPC)	(270, 270)	4 Bit	ตัวชี้ที่อยู่คำสั่งปัจจุบัน
Register (CNT)	(990, 130)	4 Bit	ตัวนับที่โปรแกรมสั่งงาน (ผลลัพธ์ของแอดเดอร์)
Adder	(380, 180)	4 Bit, +1	คำนวณ μPC + 1 (ที่อยู่ถัดไปแบบเรียงลำดับ)
Adder	(1140, 170)	4 Bit, +1	คำนวณ CNT + 1 ป้อนกลับเข้าตัวนับ
Multiplexer	(230, 240)	2 → 1, 4 Bit	เลือกที่อยู่ถัดไปคือ PC + 1 หรือ TARGET
Comparator	(1150, 330)	4 Bit	เปรียบเทียบ CNT กับค่าคงที่ 10 (ใช้เฉพาะขา A = B)
Splitter	(760, 180)	8 → 8	ผ่าคำสั่ง 8 Bit ออกเป็น Bit เดี่ยว
Splitter	(800, 90)	4 → 4	รวม Bit D3–D0 กลับเป็น Bus 4 Bit (ที่อยู่ปลายทาง)
Controlled Buffer	(1270, 300)	4 Bit	ประตู tri-state ปลอ่ยค่า CNT ออกจอเมื่อได้รับอนุญาต
Hex Digit Display	(1330, 260)	—	แสดงค่าตัวนับเป็นเลขฐานสิบหก
NOT Gate, AND, OR, NOT	(550, 420) (500, 430) (600, 440) (650, 450)	—	วงจรตัดสินใจกระโดด (Branch Logic)
Constant	(310, 190) / (1080, 180)	= 1	ค่า “+1” ป้อนให้ Adder ทั้งสองตัว
Constant	(1070, 340)	= 0xA (10)	ค่าเป้าหมายที่ใช้เปรียบเทียบ

อุปกรณ์	พิกัดในไฟล์	การตั้งค่า	หน้าที่
Constant	(250 , 320)	= 1	ผูกขา Enable ของ μ PC ไว้ตลอด $\rightarrow$ μ PC เดินทุกคาบ
Pin Reset / Clock	(170 , 380) / (170 , 510)	1 Bit	อินพุตจากผู้ใช้
Output Pin	(710 , 200) / (1300 , 300)	8 / 4 Bit	จุดสังเกตค่า (ค่าสังดับ / ค่าที่ส่งออกจอ)

4 แนวคิดการออกแบบวงจร (Design Concept)

4.1 หน่วยควบคุมสองแนวทาง : ทำไมต้องใช้ ROM เป็นหน่วยควบคุม

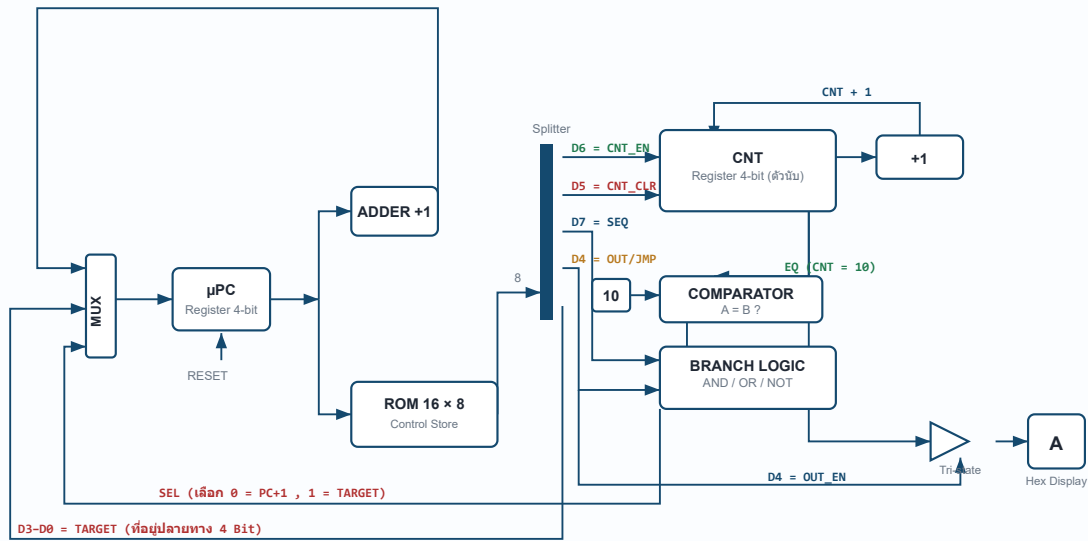
ในซีพียู หน่วยควบคุม (Control Unit) มีหน้าที่สร้างสัญญาณควบคุมไปสั่งงานส่วนต่าง ๆ เช่น สั่งให้ Register โหลดค่า สั่งให้ ALU บวก หรือสั่งให้ตัวนับเพิ่มค่า การสร้างหน่วยควบคุมสามารถทำได้ 2 แนวทาง ดังนี้

- **Hardwired Control** — ต้องวงจรลอจิกด้วย Gate โดยตรง มีข้อดีคือทำงานได้เร็ว แต่มีข้อเสียคือแก้ไขได้ยาก เพราะหากต้องการเปลี่ยนพฤติกรรมของวงจรจะต้องรี้อและต้องวงจรใหม่ทั้งหมด
- **Microprogrammed Control** — เก็บ “ลำดับของสัญญาณควบคุม” ไว้ในหน่วยความจำ (ROM) แล้วอ่านออกมาทีละค่า วิธีนี้สามารถ **เปลี่ยนพฤติกรรมของวงจรได้โดยแก้ไขเพียงข้อมูลใน ROM** ไม่ต้องแตะต้องสายไฟแม้แต่เส้นเดียว ซึ่งเป็นแนวทางที่วงจรในไฟล์นี้เลือกใช้

แต่ละแถวใน ROM เรียกว่า **Microinstruction (Microinstruction)** และตัวชี้ตำแหน่งแถวปัจจุบันเรียกว่า **μ PC (micro Program Counter)** ทั้งชุดนี้รวมกันเรียกว่า **Microsequencer (Microsequencer)**

4.2 BlockDiagram รวมของวงจร

โครงสร้างรวมของวงจรทั้งหมดแสดงไว้ในรูปที่ 1 โดยเส้นหนาสีน้ำเงินคือ Data Bus หลาย Bit ส่วนเส้นบางสีเทาคือสัญญาณควบคุมขนาด 1 Bit ส่วนสัญญาณนาฬิกา (CLK) จะต่อเข้า Register μ PC และ CNT พร้อมกัน จึงไม่ได้วาดไว้ในรูปเพื่อลดความรก



รูปที่ 1 — BlockDiagram รวมของวงจร ROM as Control Store

เส้นหนาสีน้ำเงินคือ Data Bus หลาย Bit เส้นบางคือสัญญาณควบคุม 1 Bit (สัญญาณนาฬิกาไม่ได้วาดไว้เพื่อลดความรก)

เส้นทางสัญญาณอ่านตามลูกศร :

µPC ใช้เป็น ที่อยู่ ป้อนให้ ROM → ROM คายคำสั่ง 8 Bit → Splitter ผ่านคำสั่งออกเป็นสัญญาณควบคุม 4 เส้น (D7-D4) และที่อยู่ปลายทาง 4 Bit (D3-D0) → สัญญาณควบคุมไปสั่งตัวนับ CNT และวงจรตัดสินใจกระโดด → ผลการตัดสินใจ (SEL) ย้อนกลับไปยัง MUX ว่าจะป้อนค่าใดกลับเข้า µPC ในขอบนาฬิกาถัดไป

4.3 รูปแบบของ Microinstruction (8 Bit)

หัวใจของวงจรอยู่ที่การ “ตีความ” ค่า 8 Bit ที่ ROM คายออกมา วงจรกำหนดความหมายของแต่ละ Bit ไว้ตามรูปที่ 2 โดยแบ่งเป็นสัญญาณควบคุม 4 Bit บน (D7-D4) และที่อยู่ปลายทางของการกระโดด 4 Bit ล่าง (D3-D0)

bit 7	bit 6	bit 5	bit 4	bit 3 ... bit 0
D7	D6	D5	D4	D3 D2 D1 D0
SEQ ไปคำสั่งถัดไป	CNT_EN นับเพิ่ม 1	CNT_CLR ล้างตัวนับ	OUT_EN / JMP แสดงผล + กระโดด	TARGET ADDRESS ที่อยู่ปลายทางของการกระโดด (0-15)

รูปที่ 2 — รูปแบบ Bit ของ Microinstruction

ตารางที่ 2 ความหมายของแต่ละ Bit ใน Microinstruction

Bit	ชื่อ	ต่อไปที่ไหน	ความหมายเมื่อมีค่าเป็น 1
D7	SEQ	อินพุต NOT Gate → AND	บังคับให้ไปคำสั่งถัดไปแบบเรียงลำดับ (PC + 1) ห้ามกระโดด ไม่ว่าเงื่อนไขใด ๆ

Bit	ชื่อ	ต่อไปที่ไหน	ความหมายเมื่อมีค่าเป็น 1
D6	CNT_EN	ขา Enable ของ Register CNT	อนุญาตให้ตัวนับโหลดค่าใหม่ $\rightarrow \text{CNT} \leftarrow \text{CNT} + 1$
D5	CNT_CLR	ขา Reset ของ Register CNT	ล้างตัวนับเป็น 0
D4	OUT_EN / JMP	ขา Enable ของ tri-state buffer และ อินพุต OR Gate	ทำงาน 2 อย่างพร้อมกัน : (1) เปิดจอให้แสดงค่า CNT (2) ถ้า D7 = 0 ด้วย จะกลายเป็นการ กระโดดแบบไม่มีเงื่อนไข
D3-D0	TARGET	อินพุตช่อง 1 ของ MUX	ที่อยู่ปลายทางของการกระโดด (0-15)

5 การทำงานของแต่ละส่วน (Detailed Operation)

5.1 ส่วนลำดับที่อยู่ : MUX + μ PC + Adder

สามชิ้นนี้ต่อกันเป็น วงรอบป้อนกลับ (Feedback Loop) ซึ่งเป็นแกนกลางของวงจรทั้งหมด

- μ PC (Register 4 Bit) เก็บที่อยู่ของคำสั่งที่กำลังทำงานอยู่ ค่าที่ขาออก (Q) ถูกส่งไป 2 ทางพร้อมกัน คือ ไปเป็นที่อยู่ของ ROM และไปเข้า Adder
- Adder (+1) รับค่า μ PC บวกกับค่าคงที่ 1 ได้ผลเป็น μ PC + 1 ซึ่งคือที่อยู่ถัดไปแบบเรียงลำดับ
- MUX 2 $\rightarrow$ 1 (4 Bit) เป็นจุดตัดสินใจ มีสองทางเลือก ได้แก่
 - ช่อง 0 $\leftarrow \mu$ PC + 1 (เดินหน้าตามปกติ)
 - ช่อง 1 $\leftarrow$ TARGET จาก ROM (กระโดด)

ขาเลือก (SEL) มาจากวงจรตัดสินใจในหัวข้อ 5.5 ผลลัพธ์ของ MUX คือค่าที่จะถูกโหลดเข้า μ PC ในขอบสัญญาณนาฬิกาถัดไป

- ขา Enable ของ μ PC ถูกผูกไว้กับค่าคงที่ 1 ตลอดเวลา แปลว่า μ PC เปลี่ยนค่าทุกขอบสัญญาณนาฬิกาเสมอ ไม่มีการหยุดค้าง การ “หยุด” ของโปรแกรมจึงต้องทำด้วยการ กระโดดหาตัวเอง ไม่ใช้การหยุดสัญญาณนาฬิกา
- ขา Reset ของ μ PC ต่อกับปุ่ม Reset ภายนอก ใช้บังคับให้เริ่มที่ address 0

5.2 ROM — ตัวคลังคำสั่ง

ROM ตั้งค่าเป็น address 4 Bit $\times$ data 8 Bit จึงมี 16 แถว แถวละ 8 Bit ทำงานแบบ อะซิงโครนัส (Asynchronous) คือไม่ต้องรอสัญญาณนาฬิกาทันทีที่ μ PC เปลี่ยนค่า ROM จะคายข้อมูลแถวนั้นออกมาทันที ค่าที่คายออกมานี้เองที่กลายเป็น “สัญญาณควบคุมของคาบปัจจุบัน”

5.3 Splitter — ตัวถอดรหัสคำสั่ง

Splitter ตัวใหญ่ (8 $\rightarrow$ 8) ผ่าคำสั่ง 8 Bit ออกเป็นเส้นเดี่ยว ๆ วงจรดึงไปใช้เพียง 4 เส้นบน คือ D7, D6, D5 และ D4

จุดที่มักมองข้าม — Bit D3-D0 ไม่ได้ถูกเดินสายด้วย “เส้นลวด” แต่ Splitter ตัวเล็ก (4 $\rightarrow$ 4) ถูกวาง ให้ปลายขาชนกันพอดีกับ Splitter ตัวใหญ่ ใน Logisim การที่ขาสองตัวอยู่ตำแหน่งเดียวกันถือว่า “ต่อถึงกัน” แล้ว Splitter ตัวเล็กจึงทำหน้าที่รวม Bit D3-D0 กลับเป็น Bus 4 Bit ส่งไปเป็นที่อยู่ปลายทางให้ MUX (ตรวจสอบยืนยันแล้วด้วยการจำลอง : Bus ที่ได้เท่ากับ 4 Bit ล่างของคำสั่งเสมอ)

5.4 ตัวนับ CNT — “งานจริง” ที่โปรแกรมสั่ง

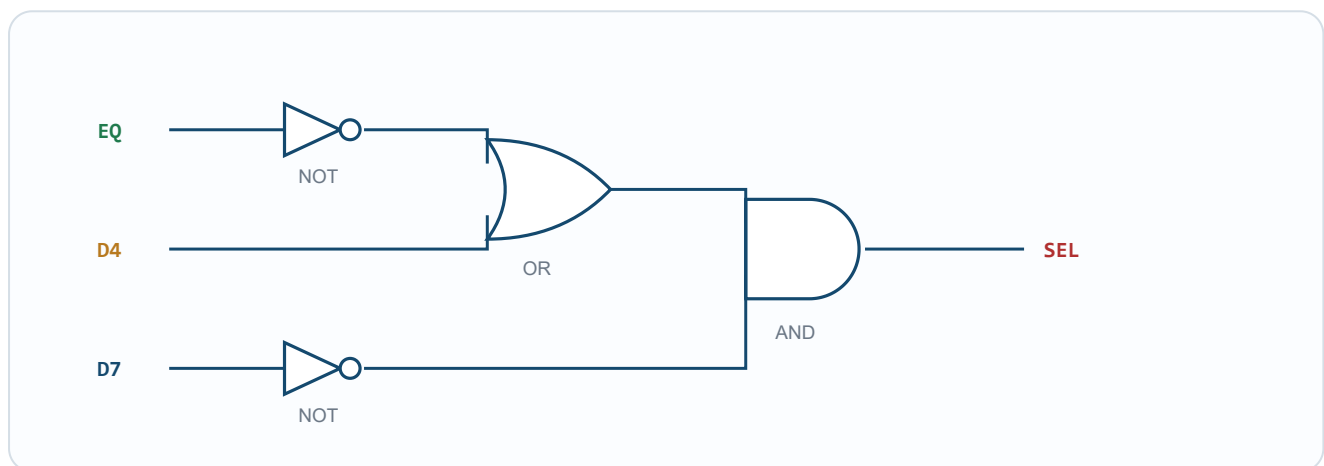
Register CNT ต่อกับ Adder (+1) แบบป้อนกลับ ทำให้เกิดเป็น **ตัวนับขึ้น (Up-counter)** ที่ *ควบคุมด้วย Software* พฤติกรรมที่ขอบสัญญาณนาฬิกา แต่ละครั้งเป็นดังนี้ (เรียงตามลำดับความสำคัญ)

ถ้า $D5 = 1 \rightarrow CNT \leftarrow 0$ | ไม่เช่นนั้น ถ้า $D6 = 1 \rightarrow CNT \leftarrow CNT + 1$ | ไม่เช่นนั้น $\rightarrow CNT$ ค้างค่าเดิม

สังเกตว่า CNT ไม่ได้ต่อกับปุ่ม Reset ภายนอกเลย การล้างค่าตัวนับต้องสั่งผ่าน **คำสั่งใน ROM (Bit D5)** เท่านั้น ซึ่งก็คือคำสั่งแรกของโปรแกรมนั่นเอง

5.5 Comparator + Branch Logic — สมอของการตัดสินใจ

Comparator เปรียบเทียบ CNT กับค่าคงที่ 10 วงจรใช้เพียงขาออกเดียวคือ **A = B** เรียกสัญญาณนี้ว่า **EQ** (EQ = 1 เมื่อ CNT นับถึง 10 พอดี) จากนั้น EQ ถูกส่งเข้า Gate 4 ตัวต่อไปนี้



รูปที่ 3 — วงจรตัดสินใจกระโดด ระดับ Gate

$$SEL = NOT(D7) \text{ AND } (D4 \text{ OR } NOT(EQ))$$

อ่านความหมายที่ละกรณีได้ตามตารางที่ 3

ตารางที่ 3 การอ่านความหมายของวงจรตัดสินใจกระโดดทีละกรณี

D7	D4	CNT = 10 ?	SEL	ผลลัพธ์
1	ใดก็ได้	ใดก็ได้	0	เดินหน้าตามลำดับ $\mu PC \leftarrow \mu PC + 1$ (D7 มีอำนาจเหนือทุกเงื่อนไข)
0	1	ใดก็ได้	1	กระโดดแบบไม่มีเงื่อนไข $\mu PC \leftarrow TARGET$
0	0	ไม่เท่า	1	กระโดด $\mu PC \leftarrow TARGET$ (ยังนับไม่ครบ $\rightarrow$ ววน Loop ต่อ)
0	0	เท่ากับ 10	0	ไม่กระโดด $\mu PC \leftarrow \mu PC + 1$ (นับครบแล้ว $\rightarrow$ ออกจาก Loop)

สองแถวล่างรวมกันคือคำสั่งชนิด “กระโดดถ้ายังไม่เท่ากับ 10” (Branch if Not Equal) ซึ่งเป็นกลไกที่ทำให้เกิด Loop ในโปรแกรม

5.6 Tri-state Buffer + Hex Display — ส่วนแสดงผล

ค่า CNT ไม่ได้ต่อตรงเข้าจอ แต่ผ่าน **Controlled Buffer (tri-state)** ที่มี Bit D4 เป็นตัวควบคุม

- **D4 = 1** → Buffer นำกระแส ค่า CNT ปรากฏบนจอ
- **D4 = 0** → Buffer อยู่ใน High Impedance state (ลอย) สายขาออกไม่มีค่า จอไม่แสดงตัวเลข

นี่คือการจำลอง **Shared Bus (Shared Bus)** ในซีพียูจริง ที่อุปกรณ์หลายตัวต่อสายเส้นเดียวกัน แต่จะมีเพียงตัวที่ได้รับ “ใบอนุญาต” จากหน่วยควบคุมเท่านั้นที่ปล่อยค่าออกมาได้ ณ เวลาหนึ่ง ๆ

5.7 สัญญาณนาฬิกาและการ Reset

- **Clock** ต่อขนาเข้าทั้ง μ PC และ CNT ทั้งสองจึงอัปเดต **พร้อมกัน** ที่ขอบขาขึ้นเดียวกัน โดยใช้สัญญาณควบคุมจากคำสั่ง **ปัจจุบัน** เป็นตัวกำหนดว่าจะเปลี่ยนค่าเป็นอะไร
- **Reset** ต่อเข้า μ PC เท่านั้น กดแล้วโปรแกรมกลับไปเริ่มที่ address 0 ส่วน CNT ถูกล้างโดยคำสั่งแรกอยู่แล้ว ผลลัพธ์สุดท้ายจึงเหมือนกับการ Reset ทั้งระบบ

6 โปรแกรมที่บรรจุอยู่ใน ROM (The Microprogram)

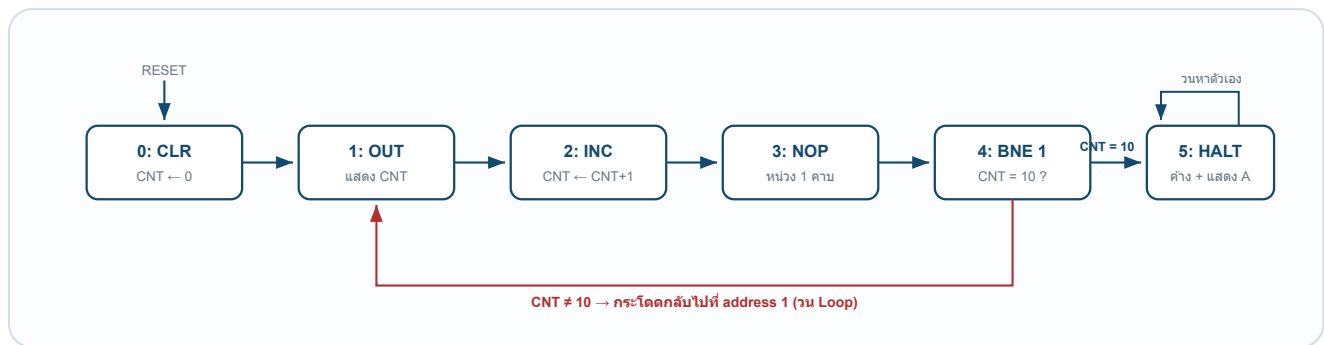
6.1 ตารางถอดรหัส Microprogram

ข้อมูลใน ROM ที่บันทึกไว้ในไฟลัจจคือ **a0 90 c0 80 01 15** (ที่อยู่ 6–15 เป็น 0 ทั้งหมด) เมื่อถอดรหัสตามรูปแบบ Bit ในหัวข้อ 4.3 จะได้โปรแกรกดังตารางที่ 4

ตารางที่ 4 การถอดรหัส Microprogram ที่บรรจุใน ROM

Addr	ข้อมูล	D7	D6	D5	D4	TARGET	เทียบเท่าคำสั่ง	คำอธิบาย
0	0xA0	1	0	1	0	0	CLR CNT	D5 = 1 → เคลียร์ตัวนับเป็น 0 ; D7 = 1 → ไปคำสั่งถัดไป
1	0x90	1	0	0	1	0	OUT	D4 = 1 → เปิด tri-state ส่งค่า CNT ออกจอ ; D7 = 1 → ไปคำสั่งถัดไป
2	0xC0	1	1	0	0	0	INC CNT	D6 = 1 → $CNT \leftarrow CNT + 1$; D7 = 1 → ไปคำสั่งถัดไป
3	0x80	1	0	0	0	0	NOP	ไม่ทำอะไร (หน่วง 1 คาบ) ; D7 = 1 → ไปคำสั่งถัดไป
4	0x01	0	0	0	0	1	BNE 1	D7 = 0, D4 = 0 → กระโดดกลับไป addr 1 ถ้า CNT $\neq$ 10 ; ถ้า = 10 ให้ไปต่อ addr 5
5	0x15	0	0	0	1	5	HALT / OUT 5	D7 = 0, D4 = 1 → กระโดดหาตัวเองตลอด (หยุด) และเปิดจอค้างค่า A

6.2 ฟังก์ชันการทำงานของโปรแกรม



รูปที่ 4 — ฟังก์ชันการทำงานของ Microprogram

อธิบายโปรแกรมเป็นภาษาคน :

- 0 — ตั้งตัวนับให้เป็น 0 ก่อน แล้วเดินหน้า
- 1 — เปิดจอแสดงค่าตัวนับปัจจุบัน แล้วเดินหน้า
- 2 — ตั้งตัวนับ +1 แล้วเดินหน้า
- 3 — คำสั่งว่าง (NOP) หน่วงเวลา 1 คาบ แล้วเดินหน้า
- 4 — ตรวจสอบว่าตัวนับถึง 10 หรือยัง : **ยังไม่ถึง** → กระโดดกลับไป address 1 (วนซ้ำ) / **ถึงแล้ว** → เดินหน้าไป address 5
- 5 — กระโดดหาตัวเองไปเรื่อย ๆ (หยุดทำงาน) พร้อมเปิดจอค้างไว้ที่ค่า A

Loop 1 → 2 → 3 → 4 → 1 ยาว 4 คาบ และวนทั้งหมด 10 รอบ เมื่อรวมกับคำสั่งตั้งค่าตอนต้น 1 คาบ จึงใช้เวลาทั้งสิ้น $1 + (10 \times 4) = 41$

ขอบสัญญาณนาฬิกา กว่าจะไปถึงคำสั่งหยุดที่ address 5

7 ตารางการทำงานที่ละคาบสัญญาณนาฬิกา (Cycle-by-Cycle Trace)

ตารางที่ 5 ได้จากการจำลองวงจรตามผังที่ถอดออกมา โดยแสดงค่า ก่อน ขอบสัญญาณนาฬิกาของแต่ละคาบ

ตารางที่ 5 การทำงานที่ละคาบสัญญาณนาฬิกา (42 คาบ)

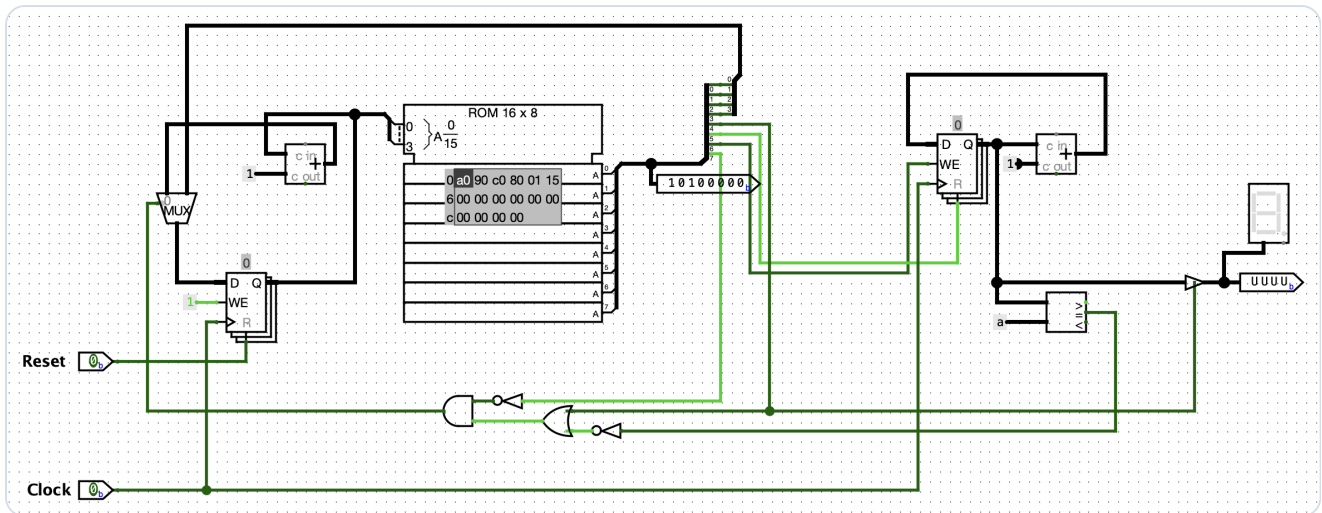
คาบ	μPC	คำสั่ง	D7	D6	D5	D4	TGT	CNT	การตัดสินใจ → μPC ถัดไป
1	0	0xA0	1	0	1	0	0	0	PC+1 → 1
2	1	0x90	1	0	0	1	0	0	PC+1 → 2
3	2	0xC0	1	1	0	0	0	0	PC+1 → 3
4	3	0x80	1	0	0	0	0	1	PC+1 → 4
5	4	0x01	0	0	0	0	1	1	BRANCH → 1
6	1	0x90	1	0	0	1	0	1	PC+1 → 2
7	2	0xC0	1	1	0	0	0	1	PC+1 → 3

คาบ	μPC	คำสั่ง	D7	D6	D5	D4	TGT	CNT	การตัดสินใจ → μPC ถัดไป
8	3	0x80	1	0	0	0	0	2	PC+1 → 4
9	4	0x01	0	0	0	0	1	2	BRANCH → 1
... วงซ้ำต่อเนื่อง (คาบ 10–36) : CNT เพิ่มขึ้นครั้งละ 1 ทุก ๆ 4 คาบ ...									
37	4	0x01	0	0	0	0	1	9	BRANCH → 1
38	1	0x90	1	0	0	1	0	9	PC+1 → 2
39	2	0xC0	1	1	0	0	0	9	PC+1 → 3
40	3	0x80	1	0	0	0	0	10	PC+1 → 4
41	4	0x01	0	0	0	0	1	10	PC+1 → 5 (นับครบ → ออกจาก Loop)
42	5	0x15	0	0	0	1	5	10	BRANCH → 5 (หยุดค้าง)

สังเกตแถวสีเน้น (address 4) — ทุกครั้งที่มาถึงจะตรวจค่า CNT トラバแต่ที่ยังไม่ถึง 10 ก็จะไม่กระโดดกลับไป address 1 จนกระทั่งคาบที่ 41 ค่า CNT = 10 พอดี เงื่อนไขเป็นเท็จ วงจรจึงปล่อยให้เดินหน้าไป address 5 และหยุดค้างอยู่ที่นั่น

8 วิธีทดลองใน Logisim-Evolution (Simulation Procedure)

- เปิดไฟล์ **Lab_1_ROM_as_Control_Store.circ** ในโปรแกรม Logisim-Evolution
- เลือกเครื่องมือ **Poke Tool** (รูปนิ้วมือ) สำหรับกดเปลี่ยนค่าที่ขาสัญญาณ
- กดที่ขา **Reset** ให้เป็น 1 แล้วกลับเป็น 0 → μPC กลับไปที่ address 0
- กดที่ขา **Clock** สลับ 0 ↔ 1 ทีละครั้ง เพื่อเดินทีละคาบ แล้วสังเกต
 - ขาออก 8 Bit = คำสั่งดิบที่ ROM คายออกมา
 - จอ Hex = ค่าตัวนับ (จะติดเฉพาะตอนอยู่ที่ address 1 และ 5)
- หากต้องการเดินอัตโนมัติ ใช้ Menu **Simulate** → **Auto-Tick Enabled** (หรือกด Ctrl/Cmd + K) แต่ **ต้องกำหนดแหล่งสัญญาณนาฬิกา** ก่อน ตามที่อธิบายไว้ในหัวข้อ 8.1 มิฉะนั้นวงจรจะไม่เดินเลย
- **ลองแก้โปรแกรม** — Right-click ที่ ROM → **Edit Contents** แล้วเปลี่ยนค่า เช่น แก่ค่าคงที่ 0xA เป็นค่าอื่นเพื่อเปลี่ยนจุดหยุดของการนับ นี่คือการประเดิมสำคัญที่สุดของแล็บนี้ : เปลี่ยนพฤติกรรมของ Hardware ได้โดยไม่ต้องแก้สายไฟ

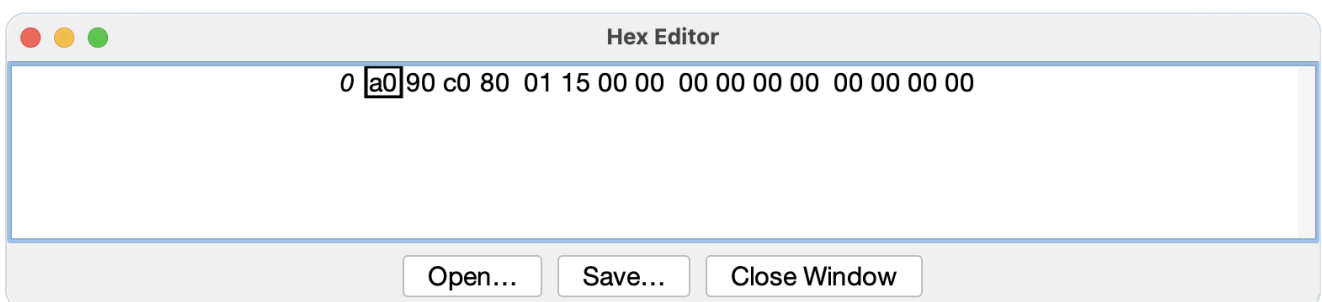


รูปที่ 5 — วงจรรวมบนพื้นที่ทำงานของ Logisim-Evolution ในสถานะเริ่มต้น ($\mu PC = 0$)

เห็น ROM 16×8 อยู่กึ่งกลางพร้อมข้อมูล a0 90 c0 80 01 15 โดยแถบไฮไลต์อยู่ที่ช่องแรก (a0)

ขาออก 8 Bit แสดงค่า 10100000 (= 0xA0) และจอ 7-segment ยังดับอยู่เพราะคำสั่งนี้มี D4 = 0

ค่าบนขาออก 8 Bit ในรูปที่ 5 คือ **10100000** ซึ่งตรงกับ 0xA0 หรือคำสั่ง **CLR CNT** ที่ address 0 ตามตารางที่ 4 ทุกประการ ส่วนจอแสดงผลยังไม่ติดเนื่องจาก Bit D4 ของคำสั่งนี้มีค่าเป็น 0 ทำให้ tri-state buffer อยู่ใน High Impedance state ตามที่อธิบายไว้ในหัวข้อ 5.6



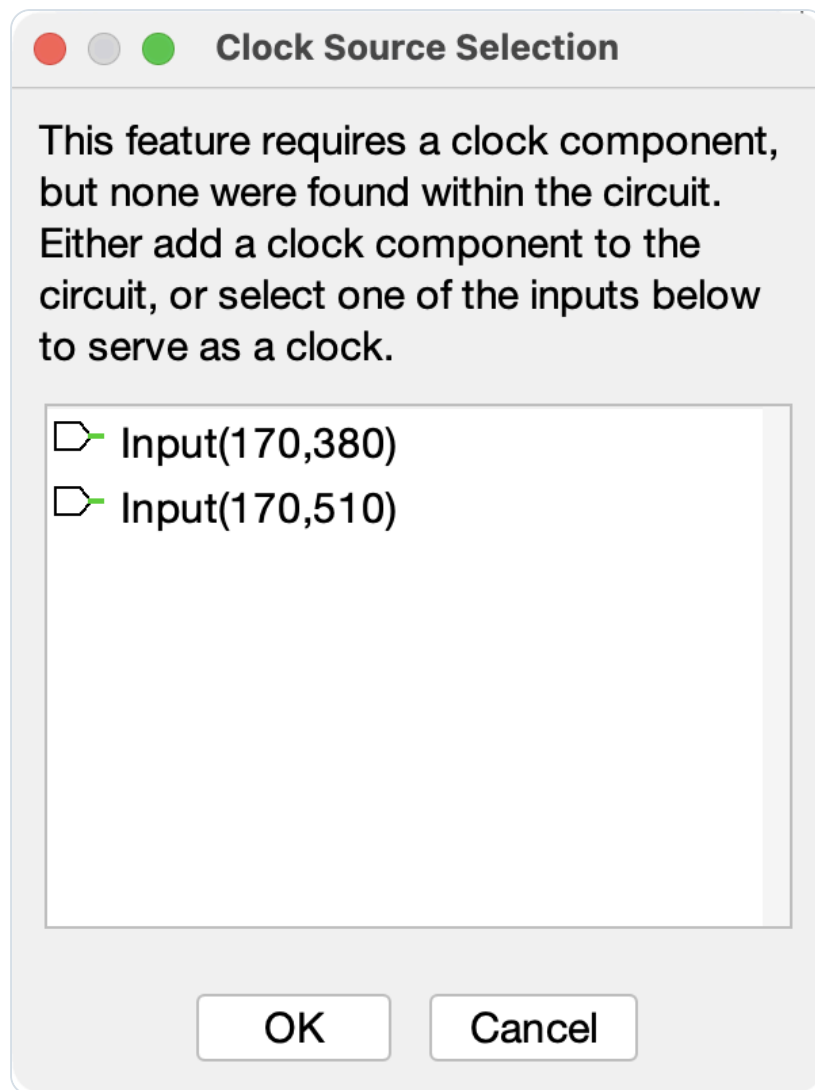
รูปที่ 6 — หน้าต่าง Hex Editor สำหรับแก้ไขข้อมูลภายใน ROM

แสดงข้อมูลครบทั้ง 16 Byte : a0 90 c0 80 01 15 แล้วตามด้วย 00 อีก 10 Byte (ที่อยู่ 6–15)

หน้าต่างนี้เปิดได้ 2 วิธี คือ Right-click ที่ ROM แล้วเลือก **Edit Contents...** หรือ Click ที่ ROM หนึ่งครั้งแล้วกดช่อง **Contents** ในแผง Properties ด้านซ้ายล่าง ข้อมูลที่ปรากฏยืนยันว่า Microprogram มีคำสั่งจริงเพียง 6 คำสั่ง ส่วนที่อยู่ 6–15 วางเปล่าเป็น 0x00 ทั้งหมด ตรงตามที่กล่าวไว้ในหัวข้อ 6.1 และข้อสังเกตในหัวข้อ 9

8.1 การกำหนดแหล่งสัญญาณนาฬิกาก่อนใช้ Auto-Tick

วงจรนี้ใช้ Pin ธรรมชาติ เป็นขา Clock ไม่ได้ใช้อุปกรณ์ **Clock component** ของ Logisim ดังนั้นเมื่อสั่ง **Simulate** → **Auto-Tick Enabled** โปรแกรมจะไม่สามารถหาสัญญาณนาฬิกาที่จะจับได้ และจะเปิดหน้าต่าง **Clock Source Selection** ขึ้นมาถามก่อนตามรูปที่ 7



รูปที่ 7 — หน้าต่าง Clock Source Selection ที่ปรากฏขึ้นเมื่อเปิด Auto-Tick

โปรแกรมแจ้งว่าไม่พบ Clock component ในวงจร และให้เลือกขาอินพุตมาทำหน้าที่แทน

จุดที่ต้องระวัง — หากปิดหน้าต่างนี้ไปโดยไม่เลือกอะไร Auto-Tick จะไม่มีผลใด ๆ วงจรจะหยุดนิ่งสนิทและดูเหมือนวงจรเสีย ทั้งที่ความจริงคือยังไม่ได้กำหนดแหล่งสัญญาณนาฬิกา

รายการที่ให้เลือกมี 2 ขา คือ **Input(170,380)** ซึ่งเป็นขา Reset และ **Input(170,510)** ซึ่งเป็นขา Clock ต้องเลือก **Input(170,510)** แล้วกด OK เท่านั้น วงจรจึงจะเดินอัตโนมัติได้ถูกต้อง

การทดลองนี้แสดงให้เห็นการสร้าง **หน่วยควบคุมแบบ Microprogram (Microprogrammed Control Unit)** ขนาดเล็กที่ใช้ ROM ขนาด 16×8 Bit ทำหน้าที่เป็นคลังคำสั่งควบคุม แทนการต่อวงจรลอจิกควบคุมด้วย Gate แบบตายตัว วงจรประกอบด้วย Microsequencer (MUX + μPC + Adder) ที่ทำหน้าที่ลำดับที่อยู่คำสั่ง ROM ที่ทำหน้าที่จ่ายสัญญาณควบคุม Splitter ที่ทำหน้าที่ถอดรหัส และวงจรตัดสินใจกระโดดที่สร้างจาก Gate เพียง 4 ตัว ตามสมการ $SEL = NOT(D7) AND (D4 OR NOT(EQ))$

ผลการจำลองการทำงานยืนยันว่าวงจรทำงานได้ถูกต้องตามที่ออกแบบไว้ กล่าวคือ โปรแกรมในหน่วยความจำสามารถสั่งให้ตัวนับล้นค่า แสดงผล เพิ่มค่า และวน Loop ตรวจสอบเงื่อนไขได้ครบถ้วน โดยไล่แสดงตัวเลข 0 ถึง 9 บนจอแสดงผล 7-segment แล้วหยุดค้างที่ค่า A (เท่ากับ 10 ในฐานสิบ) เมื่อเงื่อนไขการเปรียบเทียบเป็นจริง รวมใช้เวลาทั้งสิ้น 41 ขอบสัญญาณนาฬิกาตรงตามที่คำนวณไว้จากโครงสร้าง $Loop\ 1 + (10 \times 4)$

ข้อสรุปสำคัญที่สุดของการทดลองคือ **การเปลี่ยนพฤติกรรมของวงจร Hardware สามารถทำได้โดยแก้ไขเพียงข้อมูลใน ROM ไม่จำเป็นต้องรื้อหรือต่อสายไฟใหม่แม้แต่เส้นเดียว** ซึ่งเป็นข้อได้เปรียบหลักของหน่วยควบคุมแบบ Microprogram เหนือแบบ Hardwired Control และเป็นแนวคิดพื้นฐานที่ใช้จริงในการออกแบบหน่วยประมวลผลสมัยใหม่ นอกจากนี้การทดลองยังทำให้เข้าใจการใช้งาน Tri-state Buffer ในการจำลอง Shared Bus และเห็นความสำคัญของการออกแบบเงื่อนไขการเปรียบเทียบให้ครอบคลุม เพื่อป้องกันการวน Loop ไม่รู้จบดังที่กล่าวไว้ในข้อสังเกต

1. Mano, M. M., & Ciletti, M. D. (2018). *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog* (6th ed.). Pearson Education.
2. Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design: The Hardware/Software Interface* (6th ed.). Morgan Kaufmann.
3. Wilkes, M. V. (1951). The Best Way to Design an Automatic Calculating Machine. *Report of the Manchester University Computer Inaugural Conference*, 16–18. (ต้นกำเนิดแนวคิด Microprogrammed Control)
4. Logisim-evolution Developers. (2024). *Logisim-evolution: Digital logic designer and simulator* (Version 4.1.0) [Computer software]. สืบค้นจาก <https://github.com/logisim-evolution/logisim-evolution>
5. Tanenbaum, A. S., & Austin, T. (2013). *Structured Computer Organization* (6th ed.). Pearson Education.